

WireGuard Management and Sales Service

Setup, Nodes, Interfaces, and the Commercial API

For	System operators, Node administrators, and storefront or bot developers
Components	REST API + Worker + per-Node Agent + Persian operations console
Document scope	Describes the code that exists today, not planned features

Important scope note

This version provides the commercial API foundation, Node and Interface management, and basic operations. It is not yet feature-equivalent to WG_Panel. A full Peer and Subscription admin UI, production payment adapter, Telegram bot, 2FA, backups, and Traffic Control are not included.

Contents

- 1. System components and request flow
- 2. Local and Docker setup
- 3. Installing an Agent and registering Nodes and Interfaces
- 4. Using the operations console
- 5. Connecting a storefront and processing purchases
- 6. Scopes, Webhooks, and routine operations
- 7. Security, troubleshooting, and current limitations

1. System components and request flow

The service has four independently responsible components:

Central API

Handles Tenant and API-key authentication and manages Plans, Users, Orders, Payment confirmation, Subscriptions, Peers, Nodes, Interfaces, and Webhooks. JSON responses include success, data, and request_id.

Worker

Runs queued work outside the request cycle: provisioning and revoking Peers through Agents, polling usage, enforcing traffic limits, expiring Subscriptions, and retrying Webhook delivery. Keep it running continuously.

Agent on each Node

Authenticates with a per-Node Bearer token and runs WireGuard commands against configured interfaces. It stores Peer keys and encrypted configurations in local SQLite so it can restore them after restart.

Persian operations console

The current console is served at /panel and manages Nodes and Interfaces. Plans, Users, Orders, Subscriptions, and Peers are managed through the REST API and /docs.

Node and Interface model

A Node represents one VPS, its HTTPS Agent URL, and its credential. Each WireGuard interface has a separate Interface record with its OS interface name, country, endpoint, server public key, DNS, and address pool. For backwards-compatible sales APIs, Interface records live under /servers.

Client lifecycle

- The storefront creates a User and Order, then verifies a real payment on its own backend.
- After payment confirmation, the service creates a Subscription and queues Peer provisioning.
- The Worker contacts the Node Agent. The Peer becomes active and a WireGuard configuration is generated.
- The storefront follows status through the API or Webhooks, then provides the config, QR code, or ZIP to the customer.

2. Local and Docker setup

Run locally

Python 3.11 or newer is required. From the project directory:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
cp .env.example .env
```

Edit .env. Generate a valid Fernet key for WG_CONFIG_ENCRYPTION_KEY with:

```
python -c 'from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())'
```

Set WG_BOOTSTRAP_API_KEY to a long random value, for example from openssl rand -hex 32. This is a full-access administrative credential; never share it with customers or browser clients.

```
alembic upgrade head
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Console: <http://127.0.0.1:8000/panel> | API docs: <http://127.0.0.1:8000/docs> | Health: <http://127.0.0.1:8000/health>

Run the Worker separately

In a second terminal, using the same `.env` and database:

```
source .env/bin/activate
python -m app.worker
```

The Worker is required

Without a running Worker, Orders can be created but Peers will not be provisioned, usage will not be polled, Subscriptions will not expire, and Webhooks will not be delivered.

Run with Docker Compose

Set secure values for `POSTGRES_PASSWORD`, `WG_BOOTSTRAP_API_KEY`, and `WG_CONFIG_ENCRYPTION_KEY` in `.env`, then run:

```
docker compose up -d --build
docker compose logs -f api worker
```

Compose runs PostgreSQL, the API, and the Worker. The API container applies migrations before startup. The API is published only on `127.0.0.1:8000`; use a TLS reverse proxy for external access.

Useful commands

```
docker compose ps
docker compose logs -f worker
docker compose down
```

`docker compose down` preserves the database volume. Do not remove volumes during routine operations; doing so deletes persisted database data.

3. Installing an Agent and registering Nodes and Interfaces

Node prerequisites

- Linux with WireGuard installed and each interface already active, such as `wg0` or `wg1`.
- Configure IP forwarding, routing, NAT, and firewall rules for your network before connecting the Agent.
- The Agent does not install WireGuard or create OS interfaces, NAT, or firewall rules.

Agent credentials and service

Generate a long random `WG_AGENT_TOKEN` for each VPS. Generate a separate valid Fernet key for `WG_AGENT_STATE_KEY`. Put both in `/etc/wg-agent.env` and restrict that file to root.

```
WG_AGENT_TOKEN=<random-long-token>
WG_AGENT_STATE_KEY=<fernet-key-for-this-node>
WG_AGENT_INTERFACE=wg0
WG_AGENT_STATE_PATH=/var/lib/wg-agent/agent.sqlite3
```

`deploy/wg-agent.service` is the project's systemd template. After installing the app under `/opt/wireguard-service` and configuring `EnvironmentFile`, enable the service:

```
sudo install -m 600 deploy/wg-agent.env.example /etc/wg-agent.env
# Replace every example value with a real secret first
sudo systemctl daemon-reload
sudo systemctl enable --now wg-agent
sudo systemctl status wg-agent
```

Do not use example secrets

The environment template contains placeholders. Replace every token and key before starting the Agent or exposing any route.

TLS and network exposure

The Agent listens on 127.0.0.1:8787. Put Caddy or Nginx with HTTPS in front of it and do not expose port 8787 directly to the Internet. The central API must reach the Agent through HTTPS.

Register a Node and Interface

- Open /panel and provide an administrative API Key with servers:read and servers:write scopes.
- Create a Node with a name, two-letter country code, HTTPS Agent URL, and the matching WG_AGENT_TOKEN.
- Test the Node connection. The check validates the Bearer credential and an active target interface.
- Add an Interface with the OS interface name, display name, country, public endpoint, server public key, address pool, and DNS.

The interface_name must match `wg show` exactly. Choose a non-overlapping pool such as 10.44.0.0/24. The endpoint must resolve to a public address and UDP port reachable by clients.

Create Node and Interface through the API

```
POST /api/v1/nodes
X-API-Key: <management-key>
Content-Type: application/json

{"name":"Frankfurt-1","country":"DE","agent_url":"https://node-1.example.com","agent_secret":
"<random-long-token>"}

POST /api/v1/servers
X-API-Key: <management-key>
Content-Type: application/json

{"node_id":"node_...", "interface_name":"wg0", "name":"DE Premium", "country":"DE", "endpoint":"vpn.
example.com:51820", "public_key":"<server-public-key>", "address_pool":"10.44.0.0/24", "dns":"1.1.
1.1"}
```

Disabling a Node or Interface prevents new allocations. It does not disconnect existing Peers; their usage continues to be monitored.

4. Using the operations console

Sign in

Open /panel and enter an API Key with servers:read and servers:write. The key is kept in sessionStorage for that browser tab and is not stored in the panel database. Closing the tab clears that session.

Overview

The page shows counts for Nodes, online Nodes, and Interfaces, followed by the Node and Interface tables. Use the refresh button to reload data from the API.

Nodes

- Creating a Node sends its credential once. The server stores it encrypted with WG_CONFIG_ENCRYPTION_KEY.
- Test connection updates the Node's Online/Offline status.
- Disabling a Node prevents it from receiving new Peer assignments; existing Peers remain connected.

Interfaces

- A Node can have one or more OS-level WireGuard interfaces.
- Each Interface has an independent address pool used for Peer allocation.
- Disabling an Interface removes it from new allocation choices.
- Interface health is updated by the Worker's traffic polling.

Console scope

The current UI covers Node and Interface operations only. Manage Plans, Users, Orders, Subscriptions, Peers, and Webhooks using the REST API; /docs provides interactive endpoint documentation.

Issue a restricted key for console use

```
python -m app.cli issue-key local operator --scopes servers:read servers:write
```

The raw key is printed once. Store it in a password manager. If it is lost, issue a replacement and revoke the old credential.

5. Connecting a storefront and processing purchases

Your storefront or bot backend calls the API with a key belonging to its Tenant. Send the credential in X-API-Key. Tenant identity comes from the key; never accept a tenant ID from the request body.

Step 1: Create a Tenant and storefront key

```
python -m app.cli create-tenant shop-a "Shop A"
python -m app.cli issue-key shop-a checkout --scopes plans:read users:write orders:create
orders:read subscriptions:read subscriptions:renew peers:read
```

List valid scopes with `python -m app.cli scopes`. Give separate integrations separate keys with only the permissions they need.

Step 2: Create a User and read Plans

```
POST /api/v1/users
{"external_id":"telegram:123456"}

GET /api/v1/plans
```

Use a stable external_id from your own system. A repeated request for the same ID returns the existing User.

Step 3: Create an Order

```
POST /api/v1/orders
X-API-Key: <shop-key>
Idempotency-Key: checkout-2026-0001
Content-Type: application/json

{"user_id":"usr_...", "plan_id":"plan_...", "peer_count":1, "country":"DE"}
```

Use Idempotency-Key when retrying a request so one checkout does not create duplicate Orders. The response includes the Order and its price snapshot.

Step 4: Verify payment, then confirm it

Take payment through your provider on your own backend. Verify the provider signature or transaction status server-side. Only after successful verification, call the confirmation endpoint using a key with payments:confirm:

```
POST /api/v1/orders/{order_id}/confirm-payment
X-API-Key: <private-payment-callback-key>
```

```
{"provider": "your-gateway", "provider_reference": "txn-7781", "amount_minor": 1200, "currency": "USD"}
```

Payment security

This endpoint does not independently verify a transaction with a bank or payment provider. Never give payments:confirm to a browser, mobile app, or public bot. Keep it on a trusted backend that verifies the transaction first.

Step 5: Follow provisioning and deliver the config

- Payment confirmation returns HTTP 202: the request is accepted and provisioning runs asynchronously.
- Follow Subscription status at /api/v1/subscriptions/{id}; use Webhooks for status changes.
- After activation, list Peers at /api/v1/subscriptions/{id}/peers.
- Download a config at /api/v1/peers/{peer_id}/config, a QR code at /qr, or all active configs as a ZIP.

6. Scopes, Webhooks, and routine operations

Common scopes

- plans:read / plans:write - read and manage Plans.
- users:read / users:write - read or create/update Users.
- orders:create / orders:read - create and inspect Orders.
- payments:confirm - confirm a payment after trusted verification.
- subscriptions:read / subscriptions:write / subscriptions:renew - inspect, suspend/resume, and renew.
- peers:read / peers:write - retrieve config, QR, usage, create, and revoke Peers.
- servers:read / servers:write - manage Nodes and Interfaces; required by the current console.
- webhooks:read / webhooks:write - register destinations and inspect deliveries.
- keys:write and audit:read - rotate credentials and inspect audit records.

Subscription lifecycle and limits

A Plan defines traffic_limit_bytes, duration_days, peer_limit, price, currency, and allowed countries. Peer counters are polled and accumulated per Subscription. Quota exhaustion and expiration enqueue Peer revocation. Suspending a Subscription revokes its Peers; resuming queues provisioning again.

Webhooks

Register an HTTPS public destination and event list through POST /api/v1/webhooks. The raw secret appears only in the creation response. Deliveries use HMAC-SHA256. Headers: X-WG-Timestamp, X-WG-Signature, X-WG-Event, and X-WG-Delivery. Private/local destinations are rejected.

```
signature = HMAC_SHA256(secret, timestamp + "." + raw_body)
X-WG-Signature: sha256=<hex-digest>
```

Receivers should validate the timestamp to limit replay and compare the signature against the raw request body using a constant-time comparison. Failed deliveries are retried; inspect them at /api/v1/webhooks/{id}/deliveries.

Errors and diagnostics

- 401: missing or invalid API key; check X-API-Key.

- 403: the key is valid but lacks the endpoint's required scope.
- 404: the ID is unknown or belongs to another Tenant.
- 409: the current resource state does not allow the operation, or an idempotency conflict occurred.
- 422: invalid request body or field.
- 5xx or provisioning retries: inspect Worker logs, Agent TLS, interface name, address pool, and ``wg show <interface>``.

Use `request_id` and the X-Request-ID response header to trace requests. Config downloads are private and use Cache-Control: no-store.

7. Security, troubleshooting, and current limitations

Production security checklist

- Use a valid, random `WG_CONFIG_ENCRYPTION_KEY` and keep an encrypted backup. If it is lost, stored secrets and configs cannot be decrypted.
- Use the bootstrap key only for initial setup, then remove it from the Production environment.
- Issue separate, least-privilege API keys per Tenant and integration. Raw API keys are not stored in the database.
- Serve the API and Agent only over HTTPS. Keep the Agent on loopback and do not expose its port directly.
- Never place Agent or Webhook secrets in logs, tickets, or Git. Restrict the Agent environment file to root.
- Verify payments server-side and enforce rate limits at a reverse proxy or API gateway.
- Back up the database and `/var/lib/wg-agent/agent.sqlite3` using an independent, encrypted process; test recovery.

Troubleshooting checklist

- API will not start: inspect ``docker compose ps``, API logs, PostgreSQL health, and migration status.
- Node is Offline: check DNS/TLS, reverse proxy routing, Bearer token, and the Agent's local port.
- Interface is not found: `interface_name` must exactly match ``wg show``; confirm the Agent can execute ``wg``.
- Peer is not provisioned: confirm the Worker is running; Node and Interface are active/healthy; pool, endpoint, and public key are correct.
- Usage is stale: inspect the Worker. Usage polling runs about every 30 seconds.
- Webhook is not delivered: verify public HTTPS DNS, 2xx response, and the receiver's signature verifier.

Current version boundaries

The current implementation includes the commercial API foundation for Plans, Orders, Subscriptions, Peers, a Worker, an Agent, Node/Interface management, and a basic operations console. The console is not a full Peer or Plan admin panel. A payment-provider adapter and transaction verification must be implemented separately. Reseller wallets, Telegram bot, TOTP/2FA, public subscription page, multi-location Subscriptions, profiles, built-in backup/restore, Traffic Control, Geo blocking, and Security Monitor are not implemented. Run only one Worker replica in Production; global locking for expiry and Webhook delivery is not yet implemented.

Reference

This guide describes the current source tree. The feature reference discussed during design is `WG_Panel`:

https://github.com/Azumi67/WG_Panel

Recommended rollout

Start with a staging PostgreSQL database and one test Node. Exercise payment confirmation, Agent outages, retries, quotas, and expiry end-to-end. Use an independent process for database backups until built-in backup/restore is implemented. Replace all development keys before Production.

Quick purchase flow

Send every request from your storefront backend with the Tenant API Key. Confirm a payment only after provider verification, then deliver a config only after the Peer is active.

```
POST /api/v1/users -> {"external_id":"store-user-42"}
GET /api/v1/plans
POST /api/v1/orders + Idempotency-Key: order-42
POST /api/v1/orders/{order_id}/confirm-payment
GET /api/v1/subscriptions/{subscription_id}
GET /api/v1/subscriptions/{subscription_id}/peers
GET /api/v1/peers/{peer_id}/config or /qr
```

Delivery rule

Creating an Order does not mean it is paid. Confirming payment does not mean the tunnel is immediately active. Verify payment first, follow the provisioning status, and deliver a config only for an active Peer.